

DEPARTEMENT MATHÉMATIQUES ET INFORMATIQUE

Compte rendu du Projet Pratique

**Filière :
GLSID-ICCN**

CanaryFS : Système de surveillance par fichiers leurres pour la détection et la traçabilité des accès non autorisés

Réalisé par :

Douae TAHIRI

Hamza BELAZRI

Youssef SABRI

Mohamed AIT EL KADI

Encadré par :

M. OUAGUID

Année Universitaire : 2025 - 2026

ENSET, Avenue Hassan II - B.P. 159 - Mohammedia - Maroc

 05 23 32 22 20 / 05 23 32 35 30 – Fax : 05 23 32 25 46 - Site Web: www.enset-media.ac.ma

E-Mail : contact@enset-media.ac.ma

Remerciements

Nous tenons à adresser nos plus sincères remerciements à notre professeur, Monsieur Ouaguid Abdellah, pour sa disponibilité, ses précieux conseils, ses explications ainsi que les connaissances qu'il nous a transmises tout au long du module des Systèmes d'Exploitation. Grâce à son soutien et à ses orientations, nous avons pu mieux comprendre plusieurs notions importantes liées aux systèmes d'exploitation et à la programmation sous Linux.

Nous remercions également tous les enseignants du module des Systèmes d'Exploitation pour les connaissances et les compétences qu'ils nous ont transmises durant notre formation.

Nous remercions également l'ENSET Mohammedia pour les ressources pédagogiques mises à notre disposition afin de réaliser ce travail dans de bonnes conditions.

Enfin, nous remercions toutes les personnes ayant contribué de près ou de loin à l'aboutissement de ce projet.

Table des matières

Remerciements

Introduction générale

Chapitre 1 : Contexte général et analyse des besoins

I. Problématique

II. Objectifs du projet

III. Solutions proposées

Chapitre 2 : Architecture et conception du programme

I. Structure générale du programme

II. Arborescence des fichiers

III. Description des modules

Chapitre 3 : Fonctionnalités du programme

I. Options du programme

II. Types de canaries

III. Modes d'exécution

IV. Système de logging

V. Système d'alerte

Chapitre 4 : Implémentation technique

I. Technologies utilisées

II. Gestion des erreurs et redirection des sorties

III. Gestion des processus et threads

Chapitre 5 : Tests et validation

I. Scénario Light

II. Scénario Medium

III. Scénario Heavy

IV. Comparaison des modes

Conclusion générale

Introduction générale

L'évolution rapide des systèmes informatiques et l'augmentation des menaces de sécurité rendent la protection des données et la surveillance des activités suspectes de plus en plus importantes. Dans ce contexte, les systèmes de détection et les techniques de honeypot jouent un rôle essentiel dans l'identification des accès non autorisés et des comportements malveillants au sein d'un système informatique.

Dans le cadre du module des Systèmes d'Exploitation, nous avons réalisé un projet intitulé « CanaryFS », dont l'objectif principal est de mettre en place un système de surveillance basé sur des fichiers leurres appelés canary files. Ces fichiers sont volontairement créés dans un répertoire cible afin d'attirer l'attention d'un utilisateur ou d'un processus suspect. Toute tentative d'accès à ces fichiers déclenche automatiquement une alerte et permet d'enregistrer plusieurs informations importantes telles que l'utilisateur concerné, le processus utilisé, le PID ainsi que la date et l'heure de l'événement.

Ce projet nous a permis de mettre en pratique plusieurs notions étudiées en cours, notamment la programmation Shell sous Linux, la gestion des processus et des threads, ainsi que l'utilisation des outils de surveillance du système de fichiers comme inotifywait et fanotify. Il représente également une première approche des mécanismes utilisés dans le domaine de la cybersécurité et de la détection d'intrusions.

Ce rapport présente en détail l'architecture du programme, les fonctionnalités implémentées, les résultats des tests ainsi que les difficultés rencontrées.

Chapitre 1 : Contexte général et analyse des besoins

I. Problématique

Dans un environnement Linux, les administrateurs système et les professionnels de la cybersécurité font face à plusieurs défis liés à la protection et à la surveillance des systèmes. La détection des intrusions en temps réel reste particulièrement complexe, car des attaquants peuvent accéder à des fichiers sensibles tels que les clés SSH, les fichiers de configuration ou encore les identifiants de bases de données sans laisser de traces immédiates visibles.

En outre, les outils de surveillance existants sont souvent lourds, difficiles à configurer et peuvent consommer des ressources importantes du système, ce qui limite leur utilisation dans des environnements légers ou pédagogiques. Par ailleurs, lorsqu'une intrusion est détectée, les informations fournies ne sont pas toujours suffisantes pour une analyse forensique complète, notamment en ce qui concerne l'identification du processus, de l'utilisateur responsable ou encore l'horodatage précis de l'événement.

De plus, les solutions de sécurité traditionnelles comme les antivirus ou les pare-feu ne sont pas toujours efficaces contre les actions malveillantes réalisées par des utilisateurs internes ayant des accès légitimes au système. Face à ces limites, il devient nécessaire de disposer d'un outil simple, léger et efficace, capable de surveiller les fichiers sensibles en temps réel, de déclencher des alertes immédiates en cas d'accès suspect, de collecter des informations forensiques exploitables et de s'adapter facilement à différents environnements sans surcharger le système.

II. Objectifs du projet

L'objectif principal de ce projet est de développer un outil Bash appelé canaryfs qui permet de détecter les accès non autorisés à des fichiers sensibles sous Linux.

Le principe consiste à créer des fichiers leurres (canary files) dans un répertoire cible, puis à les surveiller en temps réel avec inotifywait et fanotify. Lorsqu'un fichier est ouvert ou modifié, le système déclenche automatiquement une alerte et enregistre des informations importantes comme le PID, le processus, l'UID, le PPID ainsi que la date et l'heure de l'événement.

Le projet propose aussi plusieurs modes d'exécution (fork, threads et subshell), ainsi que des options de contrôle pour personnaliser son utilisation. Toutes les actions et alertes sont enregistrées dans un fichier de log pour assurer une traçabilité complète du système.

III. Solutions proposées

La solution proposée dans ce projet consiste à développer un outil Bash appelé canaryfs, basé sur la création de fichiers leurres (canary files) afin de détecter les accès non autorisés à des fichiers sensibles dans un système Linux.

Le système crée automatiquement ces fichiers dans un répertoire cible, puis les surveille en temps réel à l'aide d'inotifywait et de fanotify. Toute action sur un fichier canary (ouverture, lecture ou modification) est immédiatement détectée par le système.

Lorsqu'un accès est identifié, le programme déclenche une alerte qui peut inclure une notification sur le bureau de l'utilisateur, en plus de l'enregistrement des événements dans les logs. Le système collecte également des informations forensiques importantes telles que le PID, le nom du processus, l'UID, le PPID ainsi que la date et l'heure de l'événement.

Les alertes sont enregistrées dans un fichier de log afin d'assurer une traçabilité complète, et peuvent également être affichées sous forme de notifications pour une réaction rapide de l'administrateur.

Enfin, l'outil propose plusieurs modes d'exécution (fork, threads et subshell) afin de s'adapter à différents environnements et besoins, tout en restant léger et efficace.

Chapitre 2 : Architecture et conception du programme

I. Structure générale du programme

Le programme canaryfs est structuré selon une approche modulaire. Cette organisation sépare le code en plusieurs fichiers indépendants, chacun ayant une responsabilité unique. Cette conception présente plusieurs avantages : elle facilite la maintenance, permet de tester chaque module séparément et rend le code plus lisible.

Le script principal canaryfs situé à la racine du projet constitue le point d'entrée unique. Il est responsable de l'analyse des options, de la validation des paramètres et de l'orchestration des modules. Tous les appels au programme passent obligatoirement par ce fichier.

Le script principal charge les modules via la commande source et exporte les fonctions nécessaires. Chaque module communique avec les autres via des variables globales et des fonctions exportées. Le flux d'exécution est linéaire : plantation, surveillance, alerte, puis éventuellement restauration.

Le programme dépend de plusieurs outils externes : inotifywait et fanotify pour la surveillance, lsof et /proc pour les informations forensiques, logger pour l'envoi vers syslog et notify-send pour les notifications desktop. Ces dépendances sont vérifiées au démarrage.

II. Arborescence des fichiers

```
canaryfs/
├── canaryfs                # Script principal (exécutable)
├── lib/                   # Bibliothèques de fonctions
│   ├── plant.sh           # Génération des fichiers leurres
│   ├── monitor.sh         # Surveillance (fork et subshell)
│   ├── monitor_thread.c   # Surveillance (threads en C)
│   ├── alert.sh           # Envoi des alertes
│   ├── restore.sh         # Nettoyage et restauration
│   └── log.sh             # Gestion des logs
├── canaries/              # Les fichiers leurres
│   ├── id_rsa.tpl
│   ├── env.tpl
│   ├── credentials.tpl
│   ├── shadow.tpl
│   └── backup.tpl
├── tests/                 # Scénarios de test
│   ├── test_light.sh      # Test léger (5 canaries)
│   ├── test_medium.sh     # Test moyen (50 canaries)
│   └── test_heavy.sh       # Test lourd (200 canaries)
└── canaryfs.conf          # Fichier de configuration
```

III. Description des modules

1. Module principal (canaryfs)

Ce module assure les fonctions suivantes :

- **Analyse des options** : La fonction `parse_args()` utilise `getopts` pour lire les options `-h`, `-f`, `-t`, `-s`, `-l`, `-r`, `-p`, `-a`.
- **Validation** : La fonction `validate()` vérifie la présence de `inotifywait`, du répertoire cible et son existence.
- **Compilation** : La fonction `compile_thread_monitor()` compile `monitor_thread.c` avec `gcc` et `pthread`.
- **Orchestration** : Les fonctions `run_fork()`, `run_thread()`, `run_subshell()` et `run_default()` lancent le mode correspondant.
- **Nettoyage** : Un trap capturant `EXIT`, `INT` et `TERM` assure la terminaison propre des processus enfants.

2. Module de plantation (plant.sh)

- **plant_canaries()** : Plante le nombre demandé de canaries dans le répertoire cible.
- **write_canary()** : Crée un fichier leurre. Si un template existe, il est copié. Sinon, un contenu factice est généré. Les permissions sont fixées à 600.

3. Module de surveillance (monitor.sh)

- **monitor_file()** : Surveille un seul fichier avec `inotifywait`. Chaque événement (`open`, `access`, `modify`) déclenche `_capture_forensics()`.
- **monitor_directory()** : Surveille un répertoire entier avec l'option `-r` de `inotifywait`. Vérifie que le fichier concerné est bien une canary.
- **_capture_forensics()** : Utilise `lsf` et `/proc` pour collecter `PID`, `PPID`, `processus`, `UID`. Ces informations sont transmises à `log_alert()` et `fire_alert()`.

4. Module de surveillance thread (monitor_thread.c)

Ce programme en C implémente la même logique avec des threads `pthread` et utilise `fanotify` pour une capture forensique fiable :

- **main()** : Crée un thread par fichier à surveiller via `pthread_create()`.
- **watch_file()** : Exécutée par chaque thread. Initialise un descripteur `fanotify` et lit les événements du noyau, qui contiennent directement le `PID` du processus accédant au fichier.
- **resolve_proc_info()** : Lit `/proc/<pid>/comm` et `/proc/<pid>/status` pour récupérer le nom du processus, l'`UID` et le `PPID`.
- **write_alert()** : Écrit l'alerte dans le fichier de log.

L'utilisation de `fanotify` au lieu d'`inotify` dans ce module permet de capturer le `PID` directement dans la métadonnée de l'événement, ce qui élimine la course critique qui pouvait survenir avec `lsf` pour les commandes très rapides.

5. Module d'alerte (alert.sh)

- **fire_alert()** : Vérifie si ALERT_MODE est activé. Si oui, écrit dans syslog via logger et envoie une notification desktop pour VirtualBox.

6. Module de restauration (restore.sh)

- **run_restore()** : Vérifie que l'utilisateur est root. Lit le fichier CANARY_REGISTRY et supprime chaque fichier listé. Supprime ensuite le fichier registry lui-même.

7. Module de log (log.sh)

- **init_log_dir()** : Crée le répertoire de logs et initialise le fichier history.log.
- **_log()** : Écrit une ligne au format demandé : date, nom d'utilisateur, type, message. La sortie est dirigée simultanément vers le terminal et le fichier de log via tee -a.
- **log_info(), log_error(), log_alert()** : Fonctions d'interface pour écrire respectivement des messages INFOS, ERROR et ALERT.

Chapitre 3 : Fonctionnalités du programme

I. Options du programme

Le programme canaryfs implémente huit options décrites ci-dessous.

Option -h : Affiche l'aide détaillée du programme.

Option -f : Active le mode fork. Dans ce mode, chaque fichier leurre est surveillé par un processus fils indépendant.

Option -t : Active le mode thread. La surveillance est assurée par un programme en langage C utilisant la bibliothèque pthreads et fanotify. Un thread est créé pour chaque fichier leurre.

Option -s : Active le mode subshell. Le programme s'exécute en arrière-plan comme un démon, rendant immédiatement la main à l'utilisateur.

Option -l : Permet de spécifier un répertoire personnalisé pour le stockage des fichiers de logs. Par défaut, les logs sont écrits dans /var/log/canaryfs/. Cette option est accessible aux utilisateurs normaux.

Option -r : Supprime tous les fichiers leurres plantés par le programme et restaure l'état initial du système. Cette option nécessite obligatoirement les privilèges root, conformément au cahier des charges.

Option -p : Définit le nombre de fichiers leurres à planter dans le répertoire cible. La valeur par défaut est 10. Cette option est accessible aux utilisateurs normaux.

Option -a : Active le mode alerte complet. Lorsqu'un accès à une canary est détecté, le programme écrit dans le syslog système et envoie une notification graphique sur le bureau de l'utilisateur. Cette option est accessible aux utilisateurs normaux.

II. Types de canaries

Le programme plante cinq types de fichiers leurres différents. Ces fichiers sont conçus pour imiter des fichiers sensibles réels.

id_rsa : Ce fichier imite une clé privée SSH. Son contenu factice ressemble à une vraie clé OpenSSH avec des en-têtes et des données encodées en base64. Ce type de fichier est très attractif pour un attaquant cherchant à prendre le contrôle d'un serveur.

.env : Ce fichier imite un fichier de variables d'environnement. Il contient de faux identifiants AWS, des mots de passe de base de données et des clés JWT. Les développeurs utilisent souvent ce type de fichier pour stocker des secrets, ce qui en fait une cible privilégiée.

credentials.json : Ce fichier imite un fichier de credentials cloud (AWS, GCP). Il contient un faux compte de service avec un project_id, une private_key et un client_email. Les attaquants recherchent activement ce type de fichiers sur les serveurs.

shadow.bak : Ce fichier imite une sauvegarde du fichier shadow système. Il contient de fausses entrées avec des mots de passe hachés. Ce fichier est extrêmement sensible car il permet le cassage de mots de passe.

backup.sql : Ce fichier imite une sauvegarde de base de données. Il contient une fausse table users avec des colonnes email, password_hash et api_key. Les administrateurs système sont souvent amenés à manipuler ce type de fichiers.

Le contenu de chaque fichier est généré de deux manières possibles. Si un fichier template correspondant existe dans le dossier canaries/ (comme id_rsa.tpl, env.tpl, credentials.tpl, shadow.tpl ou backup.tpl), il est copié vers la destination. Sinon, le script génère automatiquement un contenu factice approprié. Dans tous les cas, les permissions sont fixées à 600 (lecture et écriture uniquement pour le propriétaire) pour renforcer le réalisme.

III. Modes d'exécution

Le programme propose trois modes d'exécution principaux :

- **Mode fork (-f)** : Chaque fichier leurre est surveillé par un processus fils indépendant créé via la commande fork(). Ce mode offre une bonne isolation entre les surveillances mais consomme plus de ressources.
- **Mode thread (-t)** : La surveillance est assurée par un programme en langage C utilisant la bibliothèque pthreads et fanotify. Chaque fichier leurre est assigné à un thread. Ce mode est plus rapide et plus léger que le mode fork, et garantit une capture forensique fiable même pour les accès très rapides.
- **Mode subshell (-s)** : Le programme s'exécute en arrière-plan comme un démon. Le terminal est immédiatement rendu à l'utilisateur. Ce mode est idéal pour une surveillance prolongée.
- **Mode par défaut** : Aucun mode n'est spécifié, le programme surveille l'intégralité du répertoire cible de manière séquentielle.

IV. Système de logging

Par défaut, les logs sont stockés dans /var/log/canaryfs/history.log. L'option -l permet de spécifier un répertoire personnalisé pour les logs, ce qui est utile lorsque l'utilisateur ne dispose pas des privilèges root. Chaque ligne de log suit le format imposé par le cahier des charges : **yyyy-mm-dd-hh-mm-ss : username : TYPE : message**. Trois types de messages sont utilisés. Les messages INFOS pour les messages informatifs comme le démarrage du programme ou la plantation des canaries. Les messages ERROR pour les messages d'erreur comme l'absence de inotifywait ou un répertoire cible inexistant. Les messages ALERT pour les détections d'accès aux canaries. Voici un exemple concret de log :

```
2026-05-07-19-38-55 : douae-tahiri : ALERT : canary accessed - file=/home/user/.env
event=ACCESS pid=1234 process=cat uid=1000 ppid=5678
```

V. Système d'alerte

Le programme déclenche des alertes à trois niveaux différents. Premièrement, une alerte dans le terminal : lorsqu'un fichier leurre est accédé, une ligne ALERT s'affiche immédiatement dans le terminal où le programme est en cours d'exécution. Deuxièmement, une alerte dans syslog : si l'option -a est

activée, chaque alerte est également envoyée au syslog système via la commande logger. Cela permet à l'administrateur de centraliser toutes les alertes avec les autres événements système. Troisièmement, une notification desktop : la même option -a active également l'envoi d'une notification graphique sur le bureau de l'utilisateur via notify-send. Une attention particulière a été portée à l'environnement VirtualBox en exportant la variable DISPLAY=:0 pour garantir l'affichage des notifications. Ces trois niveaux d'alerte assurent une réaction rapide de l'administrateur en cas d'intrusion détectée.

Chapitre 4 : Implémentation technique

I. Technologies utilisées

Le programme s'appuie sur plusieurs technologies et outils standards de Linux :

- **Bash** : Utilisé pour le script principal, la logique métier et l'orchestration des modules. Bash est préinstallé sur toutes les distributions Linux et facilite la manipulation des fichiers et des processus.
- **C (pthreads)** : Utilisé pour la surveillance multi-thread en mode thread. Le programme en C avec la bibliothèque pthreads offre de meilleures performances que Bash pour les opérations concurrentes.
- **inotifywait** : Outil standard de Linux qui permet la détection en temps réel des accès aux fichiers. Il surveille les événements comme l'ouverture, la lecture ou la modification des fichiers. Il est utilisé dans les modes fork, subshell et par défaut.
- **fanotify** : Sous-système plus avancé du noyau Linux qui livre directement le PID du processus dans la métadonnée de l'événement. Il est utilisé dans le mode thread pour assurer une capture forensique atomique et fiable.
- **lsuf** : Commande permettant de lister les fichiers ouverts sur le système. Elle est utilisée pour collecter les informations forensiques telles que le PID, le nom du processus et l'UID de l'utilisateur ayant accédé à une canary.
- **logger** : Utilitaire qui envoie des messages au syslog système. Lorsque l'option -a est activée, chaque alerte est transmise au journal système pour une centralisation des événements.
- **notify-send** : Commande permettant d'afficher des notifications graphiques sur le bureau de l'utilisateur. Elle est utilisée avec l'option -a pour alerter visuellement l'administrateur en cas d'accès suspect.

II. Gestion des erreurs et redirection des sorties

Le programme gère activement les erreurs avec des codes spécifiques. Lorsqu'une option non existante est saisie, le code 100 est déclenché et l'aide est affichée. Si le paramètre obligatoire target_directory est manquant, le code 101 est déclenché avec affichage de l'aide. Si l'outil inotifywait n'est pas installé sur le système, le code 102 est déclenché avec un message d'erreur et la sortie du programme. Si le répertoire cible spécifié n'existe pas, le code 103 est déclenché avec affichage de l'aide. Enfin, si un utilisateur tente d'utiliser l'option -r sans les privilèges root, le code 104 est déclenché avec un message d'erreur et la sortie du programme. Après chaque erreur, le programme affiche automatiquement la documentation d'aide comme le fait l'option -h.

Conformément au cahier des charges, toutes les sorties standard (stdout) et les erreurs (stderr) sont redirigées simultanément vers le terminal et vers le fichier de log. Cette redirection est assurée dans le module log.sh par l'utilisation de tee -a sur chaque message journalisé, ce qui permet de tout capturer, y compris les messages des commandes externes comme inotifywait ou lsuf. Ainsi, l'administrateur dispose d'une traçabilité complète de toutes les actions et erreurs du programme.

III. Gestion des processus et threads

En mode fork, la fonction run_fork() parcourt la liste des canaries enregistrées dans le fichier registry et lance un processus enfant pour chaque fichier via (monitor_file "\$file") &. Chaque processus enfant

exécute indépendamment la surveillance de son fichier assigné. Le programme principal attend ensuite la fin de tous les processus avec la commande `wait`, ce qui garantit que la surveillance se poursuit tant que tous les processus sont actifs.

En mode `thread`, le programme en langage C `monitor_thread.c` est compilé et exécuté. La fonction `main()` crée un thread par fichier à surveiller via `pthread_create()`. Chaque thread exécute la fonction `watch_file()` qui initialise un descripteur `fanotify` et surveille les événements sur un fichier spécifique. Les threads partagent le même espace mémoire, ce qui rend ce mode plus léger et plus rapide que le mode `fork`.

En mode `subshell`, le programme est lancé dans un subshell en arrière-plan avec `( ... ) &`. Le PID du processus principal est affiché à l'utilisateur pour permettre son arrêt ultérieur via la commande `kill` ou `pkill`. Ce mode est idéal pour une surveillance prolongée car l'utilisateur peut continuer à utiliser son terminal normalement.

Chapitre 5 : Tests et validation

I. Scénario Light

Le scénario Light consiste à planter 5 canaries dans un seul répertoire. L'objectif de ce test est de mesurer le temps de détection de base et de comparer les performances des trois modes d'exécution (fork, thread, subshell) dans une configuration légère.

Procédure du test :

- Plantation de 5 canaries dans ce répertoire
- Lancement de la surveillance avec chaque mode
- Lecture du fichier .env pour déclencher une alerte
- Mesure du temps total d'exécution et du temps de détection

Résultats obtenus :

Mode	Temps total	Temps de détection
Fork	2014 ms	4 ms
Thread	2014 ms	2 ms
Subshell	2022 ms	8 ms

Analyse des résultats : Le mode thread est le plus rapide avec un temps de détection de seulement 2 millisecondes. Le mode fork offre un temps de détection de 4 millisecondes. Le mode subshell est légèrement plus lent avec 8 millisecondes à cause de la surcharge liée à l'exécution en arrière-plan.

II. Scénario Medium

Le scénario Medium consiste à planter 50 canaries réparties dans 5 répertoires différents (10 canaries par répertoire). L'objectif est de comparer les performances des trois modes d'exécution sous une charge modérée.

Procédure du test :

- Plantation de 10 canaries dans chaque répertoire (total 50 canaries)
- Lancement simultané de la surveillance sur les 5 répertoires
- Lecture simultanée des fichiers .env dans tous les répertoires
- Mesure du temps total d'exécution

Résultats obtenus :

Mode	Temps total
Fork	~2500 ms
Thread	~1000 ms
Subshell	~1100 ms

Analyse des résultats : Le mode thread est nettement plus performant que le mode fork avec un temps d'exécution réduit de plus de moitié (1000 ms contre 2500 ms). Le mode fork est plus lent car la création de 50 processus consomme davantage de ressources. Le mode subshell offre des performances similaires au mode thread.

III. Scénario Heavy

Le scénario Heavy consiste à planter 200 canaries réparties dans 10 répertoires différents (20 canaries par répertoire). L'objectif est de stresser le programme et d'évaluer ses performances sous charge maximale.

Procédure du test :

- Plantation de 20 canaries dans chaque répertoire (total 200 canaries)
- Lancement simultané de la surveillance sur les 10 répertoires
- Lecture simultanée des fichiers .env et id_rsa dans tous les répertoires
- Mesure du temps total d'exécution

Résultats obtenus :

Mode	Temps total
Fork	~10000 ms
Thread	~3000 ms
Subshell	~3000 ms

Analyse des résultats : Sous charge lourde, la différence entre les modes est encore plus marquée. Le mode thread est environ trois fois plus rapide que le mode fork (3000 ms contre 10000 ms). Le mode fork devient très lent car la création et la gestion de 200 processus consomment énormément de ressources système. Le mode thread reste stable et performant même avec un grand nombre de canaries.

IV. Comparaison des modes

Le tableau ci-dessous résume les performances des trois modes d'exécution pour les trois scénarios de test :

Mode	Light (5 can.)	Medium (50 can.)	Heavy (200 can.)
Fork	2014 ms	~2500 ms	~10000 ms
Thread	2014 ms	~1000 ms	~3000 ms
Subshell	2022 ms	~1100 ms	~3000 ms

Conclusion des tests :

Mode thread (-t) : Recommandé pour un grand nombre de canaries. Il offre les meilleures performances en termes de rapidité et de consommation des ressources.

Mode fork (-f) : Adapté pour un petit nombre de canaries (moins de 50). Il devient lent et gourmand en ressources au-delà.

Mode subshell (-s) : Idéal pour une surveillance prolongée en arrière-plan. Ses performances sont similaires au mode thread.

Conclusion générale

Le projet canaryfs a permis de développer un outil honeypot qui détecte les accès non autorisés aux fichiers sensibles sous Linux. Tous les objectifs du cahier des charges ont été atteints : les huit options obligatoires sont implémentées, les trois modes d'exécution (fork, thread, subshell) fonctionnent, et la gestion des erreurs est assurée par des codes spécifiques.

Ce travail a permis de mettre en pratique les concepts du module Systèmes d'Exploitation : programmation Shell, gestion des processus et threads, et utilisation d'outils standards comme inotifywait, fanotify et lsof. Plusieurs difficultés ont été surmontées, notamment les problèmes de permissions et les notifications sous VirtualBox.

canaryfs est un outil fonctionnel et performant, prêt à être utilisé en environnement réel. Le mode thread offre le meilleur temps de détection avec seulement 2 millisecondes. Ce projet constitue une base solide pour de futures évolutions.